Bài 2: tìm kiếm

Câu 6:

```
//a) cấu hình struct
typedef struct {
    char maHang[9];
    char tenHang[41];
    float gia;
} HangHoa;
// Ham nhap thong tin hang hoa
void nhapHangHoa(HangHoa hh[], int *n) {
    printf("Nhap so luong hang hoa: ");
    scanf("%d", n);
    for (int i = 0; i < *n; i++) {
        printf("Nhap thong tin hang hoa thu %d:\n", i + 1);
        printf("Ma hang: ");
        scanf("%s", hh[i].maHang);
        printf("Ten hang: ");
        scanf(" %[^\n]", hh[i].tenHang);
        printf("Gia: ");
        scanf("%f", &hh[i].gia);
    }
}
// b) Ham xuat danh sach hang hoa
void xuatHangHoa(HangHoa hh[], int n) {
    printf("\nDanh sach hang hoa:\n");
    for (int i = 0; i < n; i++) {
        printf("Ma: %s, Ten: %s, Gia: %.2f\n", hh[i].maHang, hh[i].tenHang, hh[i].gia);
    }
}
// Tim kiem tuan tu theo ma hang
int timKiemTuanTu(HangHoa hh[], int n, char ma[]) {
    for (int i = 0; i < n; i++) {
        if (strcmp(hh[i].maHang, ma) == 0)
            return i;
    }
    return -1;
}
// c) Ham sap xep danh sach hang hoa theo ten
void sapXepTheoTen(HangHoa hh[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (strcmp(hh[i].tenHang, hh[j].tenHang) > 0) {
                HangHoa temp = hh[i];
                hh[i] = hh[j];
                hh[j] = temp;
            }
        }
    }
}
```

// Tim kiem nhi phan theo ten

```
int timKiemTen(HangHoa hh[], int n, char ten[]) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        int cmp = strcmp(hh[mid].tenHang, ten);
        if (cmp == 0)
            return mid;
        else if (cmp < 0)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
```

// d) Ham sap xep danh sach hang hoa theo gia

```
void sapXepTheoGia(HangHoa hh[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (hh[i].gia > hh[j].gia) {
                HangHoa temp = hh[i];
                hh[i] = hh[j];
                hh[j] = temp;
            }
        }
    }
}
```

// Tim kiem nhi phan theo gia

```
int timKiemGia(HangHoa hh[], int n, float gia) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (hh[mid].gia == gia)
            return mid;
        else if (hh[mid].gia < gia)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
```

Bên thực hành

Tìm kiếm tuyến tính trên struct - sách

// a) Khai bao struct cho thong tin sach

```
typedef struct {
    int maSach;
    char tenSach[40];
    float gia;
} Sach;
```

// Ham nhap thong tin cac cuon sach

```
void nhapSach(Sach ds[], int n) {
    for (int i = 0; i < n; i++) {
        //printf("Nhap thong tin cuon sach thu %d:\n", i + 1);
        //printf("Ma sach: ");
        scanf("%d", &ds[i].maSach);
        //printf("Ten sach: ");
        scanf(" %[^\n]", ds[i].tenSach);
```

// Ham tim kiem cuon sach co ten Y

```
void timKiemSach(Sach ds[], int n, char tenCanTim[]) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (strcmp(ds[i].tenSach, tenCanTim) == 0) {
            xuatSach(ds[i]);
```

Tìm kiếm nhị phân trên struct - sách

// b) Khai bao struct cho thong tin sach

```
typedef struct {
    int ma_sach;
    char ten_sach[40];
    float gia;
} Sach;
void nhap_sach(Sach *s) {
    scanf("%d", &s->ma_sach);
```

```
        //printf("Gia sach: ");
        scanf("%f", &ds[i].gia);
    }
}
```

// Ham xuat thong tin cuon sach

```
void xuatSach(Sach s) {
    printf("%d - %s - %.3f\n", s.maSach, s.tenSach, s.gia);
}
```

```
            found = 1;
            break;
        }
    }
    if (!found) {
        printf("Not Found\n");
    }
}
```

```
    getchar();
    gets(s->ten_sach);
    scanf("%f", &s->gia);
}
void xuat_sach(const Sach *s) {
    printf("%d - %s - %.3f", s->ma_sach,  s->ten_sach, s->gia);
}
```

//Tìm kiem sách theo mã (thuat toán tìm kiem nhi phân)

```
int tim_sach_ma(Sach *sach, int n, int ma) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (sach[mid].ma_sach == ma) {
            return mid;
        }
        if (sach[mid].ma_sach < ma) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;  // không tìm thay sách
}
```

a)Tìm kiếm tuyến tính trên dãy số nguyên

// Hàm để nhập mảng từ người dùng

```
void nhapMang(int arr[], int n) {
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```

// Hàm tìm kiếm tuyến tính

```
int timKiemTuyenTinh(int arr[], int n, int x)
{
    for(int i = 0; i < n; i++) {
        if(arr[i] == x) {
            return i;
        }
    }
    return -1;
}
```

// Hàm in kết quả tìm kiếm

```
void inKetQua(int position) {
    if(position != -1) {
        printf("%d\n", position);
    } else {
        printf("Not Found\n");
    }
}
```

b)Tìm kiếm nhị phân trên dãy số nguyên

// Hàm nhập dữ liệu cho mảng

```
void nhapMang(int arr[], int n) {
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```

// Hàm tìm kiếm nhị phân

```
int timKiemNhiPhan(int arr[], int n, int x) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == x)
            return mid;
        if (arr[mid] < x)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
```

// Hàm in kết quả tìm kiếm

```
void inKetQua(int viTri) {
    if (viTri != -1) {
        printf("%d\n", viTri);
    } else {
        printf("Not Found\n");
    }
}
```

c)Tìm kiếm phần tử có giá trị trùng nhau với chỉ số

// Hàm nhập dữ liệu cho mảng

```
void nhapMang(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```

// Hàm kiểm tra và in phần tử trùng với chỉ số

```
void inPhanTuTrungChiSo(int arr[], int n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == i) {
            printf("%d ", arr[i]);
            found = 1;
        }
    }
    if (!found) {
        printf("Not Found");
    }
}
```

D)Tìm số lớn nhất và nhỏ hơn giá trị cho trước

// Hàm nhập dữ liệu cho mảng

```
void nhapMang(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

```
// Hàm tìm kiếm nhị phân
int binarySearch(int arr[], int low, int high, int x) {
    int result = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] < x) {
            result = arr[mid];
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return result;
}
// Hàm in kết quả tìm kiếm
void inKetQua(int result) {
    if (result == -1) {
        printf("Not Found\n");
    } else {
        printf("%d\n", result);
    }
}
```

Bài 3: Sắp xếp

Câu 3:

```
// Ham dem so phep so sanh va phep gan
void countOperations(int a[], int n, int *phepss, int *phepgan) {
    *phepss = 0;
    *phepgan = 0;
}
// Bubble Sort
void bubbleSort(int a[], int n, int *phepss, int *phepgan) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            (*phepss)++;
            if (a[j] > a[j+1]) {
                // Hoan doi
                int temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
                (*phepgan)++;
            }
        }
    }
}
```

// Selection Sort

```
void selectionSort(int a[], int n, int *phepss,
int *phepgan) {
    for (int i = 0; i < n-1; i++) {
        int minIdx = i;
        for (int j = i+1; j < n; j++) {
            (*phepss)++;
            if (a[j] < a[minIdx]) {
                minIdx = j;
            }
        }
        if (minIdx != i) {
            // Hoan doi
            int temp = a[i];
            a[i] = a[minIdx];
            a[minIdx] = temp;
            (*phepgan)++;
        }
```

// Interchange Sort

```
void interchangeSort(int a[], int n, int
*phepss, int *phepgan) {
    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            (*phepss)++;
            if (a[i] > a[j]) {
                // Hoan doi
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
                (*phepgan)++;
            }
        }
    }
}
```

// Insertion Sort

```
void insertionSort(int a[], int n, int *phepss,
int *phepgan) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            (*phepss)++;
            a[j + 1] = a[j];
            j--;
            (*phepgan)++;
        }
        a[j + 1] = key;
        (*phepgan)++;
    }
}
```

// Quick Sort

```
int partition(int a[], int low, int high, int *phepss, int *phepgan) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        (*phepss)++;
        if (a[j] < pivot) {
            i++;
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            (*phepgan)++;
        }
    }
    int temp = a[i + 1];
    a[i + 1] = a[high];
    a[high] = temp;
    (*phepgan)++;
    return i + 1;
}
void quickSort(int a[], int low, int high, int *phepss, int *phepgan) {
    if (low < high) {
        int pi = partition(a, low, high, phepss, phepgan);
        quickSort(a, low, pi - 1, phepss, phepgan);
        quickSort(a, pi + 1, high, phepss, phepgan);
    }
}
```

// Ham xuat mang

```
void xuatMang(int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}
```

Câu 4:

// Ham hoan doi gia tri cua hai bien

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

// Ham sap xep noi bot

```
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (a[j] > a[j+1]) {
                swap(&a[j], &a[j+1]);
            }
        }
    }
}
```

```
// Ham sap xep chon
void selectionSort(int a[], int n) {
    for (int i = 0; i < n-1; i++) {
        int min_idx = i;
        for (int j = i+1; j < n; j++) {
            if (a[j] < a[min_idx]) {
                min_idx = j;
            }
        }
        swap(&a[min_idx], &a[i]);
    }
}
// Ham sap xep doi cho
void interchangeSort(int a[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            if (a[i] > a[j]) {
                swap(&a[i], &a[j]);
            }
        }
    }
}
// Ham phan hoach cho Quick Sort
int partition(int a[], int low, int high) {
// Ham sap xep nhanh (Quick Sort)
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
```

```
// Ham sap xep chen
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j = j - 1;
        }
        a[j + 1] = key;
    }
}
    int pivot = a[high];
    int i = (low - 1);
    for (int j = low; j <= high-1; j++) {
        if (a[j] < pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[high]);
    return (i + 1);
}

        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}
```

```
// Ham sinh mang ngau nhien
void mangrandom(int a[], int n) {
    srand(time(0));
    for (int i = 0; i < n; i++) {
        a[i] = rand() % 100;
    }
}
```

```
// Ham in mang
void xuatmang(int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}
```

```
// Ham hien thi menu
void hienThiMenu() {
    printf("Chon phuong phap sap xep:\n");
    printf("1. Bubble Sort\n");
    printf("2. Selection Sort\n");
    printf("3. Insertion Sort\n");
    printf("4. Interchange Sort\n");
    printf("5. Quick Sort\n");
}
```

```
// Ham dem thoi gian chay cho cac ham sap xep khong co tham so high
double demThoiGian(void (*sortFunc)(int[], int), int a[], int n) {
    clock_t start, end;
    start = clock();
    sortFunc(a, n);
    end = clock();
    return ((double) (end - start)) / CLOCKS_PER_SEC;
}
```

```
// Ham dem thoi gian chay cho ham quickSort
double demThoiGianQuickSort(void (*sortFunc)(int[], int, int), int a[], int low, int high) {
    clock_t start, end;
    start = clock();
    sortFunc(a, low, high);
    end = clock();
    return ((double) (end - start)) / CLOCKS_PER_SEC;
}
```

Câu 5:

```
// Khai bao thong tin
typedef struct {
    char maSo[20];
    char hoTen[50];
    float diemTB;
} SinhVien;
// Ham nhap danh sach sinh vien
void nhapDanhSach(SinhVien sv[], int *n) {
        printf("Nhap so luong sinh vien: ");
        scanf("%d", n);
        getchar(); // Xoa ky tu xuong dong con lai trong bo dem
        for (int i = 0; i < *n; i++) {
        printf("\nNhap thong tin sinh vien thu %d:\n", i + 1);
// Ham hien thi danh sach sinh vien
void hienThiDanhSach(SinhVien sv[], int n) {
    printf("\nDanh sach sinh vien:\n");
    for (int i = 0; i < n; i++) {
        printf("Ma so: %s, Ho ten: %s, Diem TB: %.2f\n", sv[i].maSo, sv[i].hoTen, sv[i].diemTB);
    }
}
// Ham sap xep danh sach tang dan theo ho ten
void sapXepTheoHoTen(SinhVien sv[], int n) {
        printf("Ma so sinh vien: ");
        fgets(sv[i].maSo, sizeof(sv[i].maSo), stdin);
        sv[i].maSo[strcspn(sv[i].maSo, "\n")] = 0; // Xoa ky tu xuong dong
        printf("Ho ten: ");
        fgets(sv[i].hoTen, sizeof(sv[i].hoTen), stdin);
        sv[i].hoTen[strcspn(sv[i].hoTen, "\n")] = 0; // Xoa ky tu xuong dong
        printf("Diem trung binh: ");
        scanf("%f", &sv[i].diemTB);
        getchar(); // Xoa ky tu xuong dong con lai trong bo dem
    }
}
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (strcmp(sv[i].hoTen, sv[j].hoTen) > 0) {
                SinhVien temp = sv[i];
                sv[i] = sv[j];
                sv[j] = temp;
            }
        }
    }
}
```

// Ham sap xep danh sach giam dan theo diem trung binh

```
void sapXepTheoDiemTB(SinhVien sv[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (sv[i].diemTB < sv[j].diemTB) {
                SinhVien temp = sv[i];
                sv[i] = sv[j];
                sv[j] = temp;
            }
        }
    }
}
```

Bên thực hành

b)sắp xếp trên dãy phân tử kiểu struct

// Định nghĩa kiểu struct cho nhân viên

```
typedef struct {
    char maNV[10];
    char hoTen[50];
    float luong;
} NhanVien;
```

// Hàm nhập danh sách nhân viên

```
void nhapDanhSachNhanVien(NhanVien nv[], int *n) {
    printf("Nhap so luong nhan vien: ");
    scanf("%d", n);
    getchar(); // Xóa ký tự '\n' còn lại trong bộ đệm
    for (int i = 0; i < *n; i++) {
        printf("\nNhap thong tin nhan vien thu %d:\n", i + 1);
        printf("Ma nhan vien: ");
        fgets(nv[i].maNV, sizeof(nv[i].maNV), stdin);
        nv[i].maNV[strcspn(nv[i].maNV, "\n")] = '\0'; // Loại bỏ ký tự '\n'
        printf("Ho ten: ");
        fgets(nv[i].hoTen, sizeof(nv[i].hoTen), stdin);
        nv[i].hoTen[strcspn(nv[i].hoTen, "\n")] = '\0';
        printf("Luong: ");
        scanf("%f", &nv[i].luong);
        getchar();
    }
}
```

// Hàm xuất danh sách nhân viên

```
void xuatDanhSachNhanVien(NhanVien nv[], int n) {
    printf("\nDanh sach nhan vien:\n");
    printf("%-10s %-30s %-10s\n", "Ma NV", "Ho Ten", "Luong");
    for (int i = 0; i < n; i++) {
        printf("%-10s %-30s %-10.2f\n", nv[i].maNV, nv[i].hoTen, nv[i].luong);
    }
}
```

// Hàm sắp xếp giảm dần theo lương (Bubble Sort)

```
void sapXepGiamDanLuong(NhanVien nv[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (nv[j].luong < nv[j + 1].luong) {
                NhanVien temp = nv[j];
                nv[j] = nv[j + 1];
                nv[j + 1] = temp;
            }
        }
    }
    printf("\nDanh sach sau khi sap xep giam dan theo luong:\n");
    xuatDanhSachNhanVien(nv, n);
}
```

// Hàm sắp xếp tăng dần theo mã nhân viên (Selection Sort)

```
void sapXepTangDanMaNV(NhanVien nv[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (strcmp(nv[j].maNV, nv[minIndex].maNV) < 0) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            NhanVien temp = nv[i];
            nv[i] = nv[minIndex];
            nv[minIndex] = temp;
        }
    }
    printf("\nDanh sach sau khi sap xep tang dan theo ma nhan vien:\n");
    xuatDanhSachNhanVien(nv, n);
}
```

// Hàm hiển thị menu

```
void hienThiMenu() {
    printf("\n========== MENU ==========\n");
    printf("1. Nhap danh sach nhan vien\n");
```

```
    printf("2. Xuat danh sach nhan vien\n");
    printf("3. Sap xep giam dan theo luong\n");
    printf("4. Sap xep tang dan theo ma nhan vien\n");
    printf("0. Thoat\n");
    printf("Chon chuc nang: ");
}
```

Bài 4: Danh sách

Câu 1: Trên lớp

```
// Dinh nghia cau truc sinh vien
typedef struct SinhVien {
    char maSV[10];
    char tenSV[40];
    float diemTB;
    struct SinhVien* next;
} SinhVien;
// Ham tao sinh vien moi
SinhVien* taoSinhVien(char* maSV, char*
tenSV, float diemTB) {
    SinhVien* sv = (SinhVien*)
malloc(sizeof(SinhVien));
    strcpy(sv->maSV, maSV);
    strcpy(sv->tenSV, tenSV);
    sv->diemTB = diemTB;
    sv->next = NULL;
    return sv;
}
// Ham them sinh vien vao cuoi danh sach
void themSinhVien(SinhVien** head, char*
maSV, char* tenSV, float diemTB) {
    SinhVien* sv = taoSinhVien(maSV,
tenSV, diemTB);
    if (*head == NULL) {
        *head = sv;
        return;
    }
    SinhVien* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = sv;
}
```

// Ham tinh trung binh cong diem cua sinh vien co diem trung binh > 7

```
float tinhTBCDiemLonHon7(SinhVien* head) {
    int count = 0;
    float sum = 0.0;
    SinhVien* temp = head;
    while (temp != NULL) {
        if (temp->diemTB > 7.0) {
            sum += temp->diemTB;
            count++;
        }
        temp = temp->next;
    }
    return (count == 0) ? 0 : sum / count;
}
```

// Ham sap xep danh sach sinh vien tang dan theo diem trung binh

```
void sapXepTangDanTheoDiemTB(SinhVien* head) {
    if (head == NULL) return;
    SinhVien* i;
    SinhVien* j;
    for (i = head; i->next != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->diemTB > j->diemTB) {
                // Hoan doi
```

// Ham tim sinh vien co diem trung binh lon nhat

```
SinhVien* timSVCoDiemTBLonNhat(SinhVien* head) {
    if (head == NULL) return NULL;
    SinhVien* maxSV = head;
    SinhVien* temp = head;
    while (temp != NULL) {
        if (temp->diemTB > maxSV->diemTB) {
            maxSV = temp;
        }
        temp = temp->next;
    }
    return maxSV;
}
```

```
                char maSVTmp[10], tenSVTmp[40];
                float diemTBTmp;
                strcpy(maSVTmp, i->maSV);
                strcpy(tenSVTmp, i->tenSV);
                diemTBTmp = i->diemTB;
                strcpy(i->maSV, j->maSV);
                strcpy(i->tenSV, j->tenSV);
                i->diemTB = j->diemTB;
                strcpy(j->maSV, maSVTmp);
                strcpy(j->tenSV, tenSVTmp);
                j->diemTB = diemTBTmp;
            } } } }
```

```
// Ham sap xep danh sach sinh vien tang dan theo ten
void sapXepTangDanTheoTen(SinhVien* head) {
    if (head == NULL) return;
    SinhVien* i;
    SinhVien* j;
    for (i = head; i->next != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (strcmp(i->tenSV, j->tenSV) > 0) {
                // Hoan doi
                char maSVTmp[10], tenSVTmp[40];
                float diemTBTmp;
                strcpy(maSVTmp, i->maSV);
                strcpy(tenSVTmp, i->tenSV);
                diemTBTmp = i->diemTB;
                strcpy(i->maSV, j->maSV);
                strcpy(i->tenSV, j->tenSV);
                i->diemTB = j->diemTB;
                strcpy(j->maSV, maSVTmp);
                strcpy(j->tenSV, tenSVTmp);
                j->diemTB = diemTBTmp;
            }
        }
    }
}

// Ham in danh sach sinh vien
void inDanhSach(SinhVien* head) {
    SinhVien* temp = head;
    while (temp != NULL) {
        printf("Ma SV: %s, Ten SV: %s, Diem TB: %.2f\n", temp->maSV, temp->tenSV, temp->diemTB);
        temp = temp->next;
    }
}
```

Câu 3: trên lớp

```
// Khai bao cau truc cua mot don thuc
typedef struct DonThuc {
    float heSo;
    int bac;
    struct DonThuc* next;
} DonThuc;
```

```
// Ham tao mot don thuc
DonThuc* taoDonThuc(float heSo, int bac)
{
    DonThuc* newNode =
(DonThuc*)malloc(sizeof(DonThuc));
    newNode->heSo = heSo;
    newNode->bac = bac;
    newNode->next = NULL;
    return newNode;
// Ham tim bac cao nhat trong da thuc
int timBacCaoNhat(DonThuc* head) {
    int maxBac = 0;
    while (head != NULL) {
        if (head->bac > maxBac) {
            maxBac = head->bac;
        }
        head = head->next;
    }
    return maxBac;
}
// Ham tinh gia tri cua da thuc tai gia tri x cho truoc
float tinhGiaTriDaThuc(DonThuc* head, float x) {
    float ketQua = 0;
    while (head != NULL) {
        ketQua += tinhGiaTriDonThuc(head, x);
        head = head->next;
    }
    return ketQua;
}
}
// Ham chen don thuc vao dau danh sach
void chenDonThuc(DonThuc** head, float
heSo, int bac) {
    DonThuc* newNode =
taoDonThuc(heSo, bac);
    newNode->next = *head;
    *head = newNode;
}
// Ham tinh gia tri cua mot don thuc tai
gia tri x cho truoc
float tinhGiaTriDonThuc(DonThuc*
donThuc, float x) {
    float ketQua = donThuc->heSo;
    int i;
    for (i = 0; i < donThuc->bac; i++) {
        ketQua *= x;
    }
    return ketQua;
}
```

```
// Ham hien thi da thuc
void hienThiDaThuc(DonThuc* head) {
    while (head != NULL) {
        if (head->bac == 0) {
            printf("%.2f", head->heSo);
        } else if (head->bac == 1) {
            printf("%.2fx", head->heSo);
        } else {
            printf("%.2fx^%d", head->heSo,
head->bac);
        }
        if (head->next != NULL) {
            printf(" + ");
        }
        head = head->next;
    }
    printf("\n");
}
```

Câu1: Phần bài tập

```
// Dinh nghia cau truc cho mot nut trong
danh sach lien ket don
typedef struct Node {
    int data;
    struct Node* next;
} Node;
// Ham tao nut moi
Node* taoNode(int data) {
    Node* newNode = (Node*)
malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
// Ham them nut vao cuoi danh sach lien
ket
void themCuoi(Node** head, int data) {
    Node* newNode = taoNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
// Ham xuat danh sach lien ket
void xuatDanhSach(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n"); }
```

// Ham tinh trung binh cong gia tri cua cac nut trong danh sach lien ket

```
float tinhTrungBinhCong(Node* head) {
    int sum = 0, count = 0;
    Node* temp = head;
    while (temp != NULL) {
        sum += temp->data;
        count++;
        temp = temp->next;
    }
    return (count == 0) ? 0 : (float)sum /
count;
}
```

// Ham tim nut co gia tri lon nhat

```
Node* timNutLonNhat(Node* head) {
    if (head == NULL) return NULL;
    Node* maxNode = head;
    Node* temp = head->next;
    while (temp != NULL) {
        if (temp->data > maxNode->data) {
            maxNode = temp;
        }
        temp = temp->next;
    }
    return maxNode;
}
```

// Ham dem so luong nut co gia tri chan

```
int demNutChan(Node* head) {
    int count = 0;
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data % 2 == 0) {
            count++;
        }
        temp = temp->next;
    }
    return count;
}
```

// Ham tach danh sach thanh hai danh sach chan va le

```
void tachDanhSach(Node* head, Node**
list_chan, Node** list_le) {
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data % 2 == 0) {
            themCuoi(list_chan, temp->data);
        } else {
            themCuoi(list_le, temp->data);
        }
        temp = temp->next;
    }
}
```

```
// Ham ghep hai danh sach tang dan
Node* ghepDanhSach(Node* list1, Node* list2) {
    Node* result = NULL;
    Node** lastPtrRef = &result;
    while (1) {
        if (list1 == NULL) {
            *lastPtrRef = list2;
            break;
        } else if (list2 == NULL) {
            *lastPtrRef = list1;
            break;
        }
        if (list1->data <= list2->data) {
            themCuoi(lastPtrRef, list1->data);
            list1 = list1->next;
        } else {
            themCuoi(lastPtrRef, list2->data);
            list2 = list2->next;
        }
        lastPtrRef = &((*lastPtrRef)->next);
    }
    return result;
}
```

Câu2: Phần bài tập

```
// Dinh nghia cau truc sinh vien
typedef struct {
    char maSV[10];
    char hoTen[50];
    float diemTB;
} SinhVien;
// Ham tao sinh vien moi
SinhVien taoSinhVien(char* maSV, char* hoTen, float diemTB) {
    SinhVien sv;
    strcpy(sv.maSV, maSV);
    strcpy(sv.hoTen, hoTen);
    sv.diemTB = diemTB;
    return sv;
}
```

```
// Dinh nghia cau truc nut trong danh sach lien ket don
typedef struct Node {
    SinhVien data;
    struct Node* next;
} Node;
// Ham tao nut moi
Node* taoNode(SinhVien data) {
    Node* newNode = (Node*) malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

```
// Ham them nut vao cuoi danh sach lien ket
void themCuoi(Node** head, SinhVien data) {
    Node* newNode = taoNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
// Ham xuat danh sach lien ket
void xuatDanhSach(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("Ma SV: %s, Ho ten: %s, Diem TB: %.2f\n", temp->data.maSV, temp->data.hoTen, temp->data.diemTB);
        temp = temp->next;
    }
}
```

Câu3: Phần bài tập

```
// Cau truc nut
typedef struct Node {
    int data;
    struct Node* next;
} Node;
// Khoi tao danh sach
void khoiTaoDanhSach(List* list) {
    list->head = NULL;
}
// Them chu so vao danh sach
void themChuSo(List* list, int digit) {
```

```
// Cau truc danh sach lien ket
typedef struct {
    Node* head;
} List;

    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = digit;
    newNode->next = list->head;
    list->head = newNode;
}
```

```
// Hien thi so lon
void hienThiSoLon(List list) {
    Node* current = list.head;
    while (current != NULL) {
        printf("%d", current->data);
        current = current->next;
    }
    printf("\n");
}
// Thuc hien phep cong hai so lon
List congSoLon(List num1, List num2) {
    List result;
    khoiTaoDanhSach(&result);
    Node *n1 = num1.head, *n2 =
num2.head;
    int carry = 0;
// Thuc hien phep tru hai so lon (gia su
num1 >= num2)
List truSoLon(List num1, List num2) {
    List result;
    khoiTaoDanhSach(&result);
    Node *n1 = num1.head, *n2 =
num2.head;
    int borrow = 0;
    while (n1 != NULL) {
        int diff = n1->data - borrow;
        if (n2 != NULL) {
            diff -= n2->data;
            n2 = n2->next;
    while (n1 != NULL || n2 != NULL || carry
> 0) {
        int sum = carry;
        if (n1 != NULL) {
            sum += n1->data;
            n1 = n1->next;
        }
        if (n2 != NULL) {
            sum += n2->data;
            n2 = n2->next;
        }
        themChuSo(&result, sum % 10);
        carry = sum / 10;
    }
    return result;
}
        }
        if (diff < 0) {
            diff += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        themChuSo(&result, diff);
        n1 = n1->next;
    }
    return result;
}
```

Câu4: Phần bài tập

```
// Dinh nghia cau truc cua mot nut
typedef struct Node {
    int he_so;
    int so_mu;
    struct Node* next;
} Node;
// Ham tao nut moi
Node* taoNode(int he_so, int so_mu) {
    Node* New_Node = (Node*)malloc(sizeof(Node));
    New_Node->he_so = he_so;
    New_Node->so_mu = so_mu;
    New_Node->next = NULL;
    return New_Node;
}
// Ham khoi tao da thuc (danh sach rong)
Node* khoiTaoDaThuc() {
    return NULL;
}
// Ham them hang tu vao da thuc
void themHangTu(Node** dau, int he_so, int so_mu) {
    Node* New_Node = taoNode(he_so, so_mu);
    if (*dau == NULL || (*dau)->so_mu < so_mu) {
        New_Node->next = *dau;
        *dau = New_Node;
    } else {
        Node* hien_tai = *dau;
        while (hien_tai->next != NULL && hien_tai->next->so_mu > so_mu) {
            hien_tai = hien_tai->next;
        }
        New_Node->next = hien_tai->next;
        hien_tai->next = New_Node;
    }
}
// Ham hien thi da thuc
void hienThiDaThuc(Node* dau) {
    Node* hien_tai = dau;
    while (hien_tai != NULL) {
        printf("%dx^%d", hien_tai->he_so, hien_tai->so_mu);
        if (hien_tai->next != NULL)
            printf(" + ");
        hien_tai = hien_tai->next;
    }
    printf("\n");
}
```

// Ham tinh gia tri da thuc tai gia tri x cho truoc

```
int tinhGiaTriDaThuc(Node* dau, int x) {
    int gia_tri = 0;
    Node* hien_tai = dau;
    while (hien_tai != NULL) {
        gia_tri += hien_tai->he_so * pow(x, hien_tai->so_mu);
        hien_tai = hien_tai->next;
    }
    return gia_tri;
}
```

Bên thực hành

//A: Nhập xuất số nguyên thêm cuối

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
void addToEnd(Node** head, int value) {
    Node* newNode = createNode(value);
    if (*head == NULL) {
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
void printList(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

//B: Nhập xuất số nguyên thêm đầu

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

```
// Hàm tạo nút mới
Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
// Hàm thêm phần tử vào đầu danh sách
void addToHead(Node** head, int value) {
    Node* newNode = createNode(value);
    newNode->next = *head;
    *head = newNode;
}
// Hàm xuất danh sách
void printList(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
//C: Chèn phần tử vào dãy số
typedef struct Node {
    int data;
    struct Node* next;
} Node;
// Hàm tạo nút mới
Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
// Hàm thêm phần tử vào cuối danh sách
void appendNode(Node** head, int value) {
    Node* newNode = createNode(value);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

```
// Hàm chèn phần tử y sau phần tử x
void insertAfter(Node* head, int x, int y) {
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data == x) {
            Node* newNode = createNode(y);
            newNode->next = temp->next;
            temp->next = newNode;
            return;
        }
        temp = temp->next;
```

```
//D: Đếm nút danh sách
// Định nghĩa cấu trúc của một nút trong danh sách liên kết đơn
typedef struct Node {
    int data;
    struct Node* next;
} Node;
// Hàm tạo một nút mới
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
```

```
// Hàm thêm một nút vào cuối danh sách liên kết
void append(Node** head, int data) {
    Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

```
    }
}
// Hàm xuất danh sách
void printList(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
    if (!newNode) {
        printf("Không thể cấp phát bộ nhớ!\n");
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

// Hàm đếm số lượng nút trong danh sách liên kết

```
int countNodes(Node* head) {
    int count = 0;
    Node* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
```

//E: Giá trị nút thứ k

// Định nghĩa cấu trúc của một nút trong danh sách liên kết

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

// Hàm tạo một nút mới

```
Node* taoNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

// Hàm thêm một nút vào cuối danh sách

```
void themNode(Node** head, int data) {
    Node* newNode = taoNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

// Hàm tìm giá trị của nút thứ k

```
int timGiaTriK(Node* head, int k) {
    Node* temp = head;
    int count = 0;
    while (temp != NULL) {
        if (count == k) {
            return temp->data;
        }
        count++;
        temp = temp->next;
    }
    return -1; // Trường hợp không tìm thấy
}
```

//F: Phần tử lớn nhất nhỏ nhất

// Định nghĩa cấu trúc của một nút trong danh sách liên kết

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

// Hàm tạo một nút mới

```
Node* taoNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

// Hàm thêm một nút vào cuối danh sách

```
void themNode(Node** head, int data) {
    Node* newNode = taoNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

// Hàm tìm giá trị nhỏ nhất trong danh sách

```
int timGiaTriNhoNhat(Node* head) {
    int min = head->data;
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data < min) {
            min = temp->data;
        }
        temp = temp->next;
    }
    return min;
}
```

// Hàm tìm giá trị lớn nhất trong danh sách

```
int timGiaTriLonNhat(Node* head) {
    int max = head->data;
    Node* temp = head;
    while (temp != NULL) {
        if (temp->data > max) {
            max = temp->data;
        }
        temp = temp->next;
    }
    return max;
}
```

//G: Xóa phần tử có khóa x

// Định nghĩa cấu trúc của một nút trong danh sách liên kết

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

// Hàm tạo một nút mới

```
Node* taoNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

// Hàm thêm một nút vào cuối danh sách

```
void themNode(Node** head, int data) {
    Node* newNode = taoNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

// Hàm xóa nút đầu tiên có giá trị bằng x

```
int xoaNode(Node** head, int x) {
    Node* temp = *head;
    Node* prev = NULL;

    // Nếu nút cần xóa là nút đầu tiên
    if (temp != NULL && temp->data == x) {
        *head = temp->next;
        free(temp);
        return 1;
    }
```

// Tìm nút cần xóa

```
    while (temp != NULL && temp->data != x) {
        prev = temp;
        temp = temp->next;
    }
```

// Nếu không tìm thấy nút cần xóa

```
    if (temp == NULL) return 0;
```

// Xóa nút

```
    prev->next = temp->next;
    free(temp);
    return 1;
}
```

```
// Hàm in danh sách liên kết
void inDanhSach(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
//H: Sắp xếp
```

```
// Định nghĩa cấu trúc của một nút trong danh sách liên kết
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

```
// Hàm tạo một nút mới
Node* taoNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

```
// Hàm thêm một nút vào cuối danh sách
void themNode(Node** head, int data) {
    Node* newNode = taoNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

```
// Hàm đổi chỗ hai giá trị
void hoanDoi(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Hàm sắp xếp danh sách liên kết theo thứ tự tăng dần sử dụng Interchange Sort
void sapXepTangDan(Node* head) {
    for (Node* p = head; p != NULL; p = p->next) {
        for (Node* q = p->next; q != NULL; q = q->next) {
            if (p->data > q->data) {
                hoanDoi(&p->data, &q->data);
            }
        }
    }
}
```

// Hàm sắp xếp danh sách liên kết theo thứ tự giảm dần sử dụng Interchange Sort

```
void sapXepGiamDan(Node* head) {
    for (Node* p = head; p != NULL; p = p->next) {
        for (Node* q = p->next; q != NULL; q = q->next) {
            if (p->data < q->data) {
                hoanDoi(&p->data, &q->data);
            }
        }
    }
}
```

// Hàm in danh sách liên kết

```
void inDanhSach(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

//A: Struct nhập xuất danh sách sinh viên

// Dinh nghia kieu du lieu sinh vien

```
typedef struct {
    char maso[11];
    char hoten[41];
    float diemtrungbinh;
} SV;
```

// Ham nhap thong tin cho mot sinh vien

```
void nhap1sv(SV *x) {
    scanf("%s", x->maso);
    getchar(); // De tranh viec nhap bi loi khi dung fgets hay gets sau do
    fgets(x->hoten, 41, stdin);
    x->hoten[strcspn(x->hoten, "\n")] = 0; // Xoa dau '\n' do fgets them vao
    scanf("%f", &x->diemtrungbinh);
}
```

// Ham xuat thong tin cua mot sinh vien

```
void xuat1sv(SV x) {
    printf("%s - %s - %.2f\n", x.maso, x.hoten, x.diemtrungbinh);
}
```

// Dinh nghia cau truc node cho danh sach lien ket don

```
typedef struct node {
    SV info;
    struct node* next;
} node;
```

// Ham khoi tao danh sach lien ket don (ban dau la NULL)

```
void init(node **phead) {
    *phead = NULL;
}
```

```
// Ham tao node moi
node* createnode(SV x) {
    node* newnode = (node*)malloc(sizeof(node));
    newnode->info = x;
    newnode->next = NULL;
// Ham them sinh vien vao dau danh sach
void insertfirst(node **phead, SV x) {
    node* newnode = createnode(x);
    newnode->next = *phead;
    *phead = newnode;
}
// Ham them sinh vien vao cuoi danh sach
void insertlast(node **phead, SV x) {
    node* newnode = createnode(x);
    if (isempty(*phead)) {
        *phead = newnode;
    } else {
        node* current = *phead;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = newnode;
    }
}
    return newnode;
}
// Ham kiem tra danh sach rong
int isempty(node* phead) {
    return phead == NULL;
}
```

// Ham xoa sinh vien khoi danh sach (dua tren ma so)

```
void deletesv(node **phead, char *maso) {
    if (isempty(*phead)) {
        printf("Danh sach rong\n");
        return;
    }
    node *current = *phead, *prev = NULL;
    while (current != NULL) {
        if (strcmp(current->info.maso, maso) == 0) {
            if (prev == NULL) { // Xoa phan tu dau
                *phead = current->next;
            } else {
                prev->next = current->next;
            }
            free(current);
            printf("Da xoa sinh vien co ma so: %s\n", maso);
            return;
        }
        prev = current;
        current = current->next;
    }
    printf("Khong tim thay sinh vien voi ma so: %s\n", maso);
}
```

// Ham xuat danh sach sinh vien

```
void xuatdanhmuc(node* phead) {
    if (isempty(phead)) {
        printf("Danh sach rong\n");
        return;
    }
    node* current = phead;
    while (current != NULL) {
        xuat1sv(current->info);
        current = current->next;
    }
}
```

//B: Struct lọc sinh viên theo điểm tb

// Dinh nghia kieu du lieu sinh vien

```
typedef struct {
    char maso[11];
    char hoten[41];
    float diemtrungbinh;
} SV;
```

// Ham nhap thong tin cho mot sinh vien

```
void nhap1sv(SV *x) {
    scanf("%s", x->maso);
    getchar(); // De tranh viec nhap bi loi khi dung fgets hay gets sau do
    fgets(x->hoten, 41, stdin);
    x->hoten[strcspn(x->hoten, "\n")] = 0; // Xoa dau '\n' do fgets them vao
    scanf("%f", &x->diemtrungbinh);
}
```

```
// Ham xuat thong tin cua mot sinh vien
void xuat1sv(SV x) {
    printf("%s - %s - %.2f\n", x.maso, x.hoten, x.diemtrungbinh);
}

// Ham khoi tao danh sach lien ket don (ban dau la NULL)
void init(node **phead) {
    *phead = NULL;
}
// Ham tao node moi
node* createnode(SV x) {
// Ham kiem tra danh sach rong
int isempty(node* phead) {
    return phead == NULL;
}
// Ham them sinh vien vao dau danh sach
// Ham them sinh vien vao cuoi danh sach
void insertlast(node **phead, SV x) {
    node* newnode = createnode(x);
    if (isempty(*phead)) {
        *phead = newnode;
    } else {
        node* current = *phead;
```

```
// Dinh nghia cau truc node cho danh sach lien ket don
typedef struct node {
    SV info;
    struct node* next;
} node;
    node* newnode = (node*)malloc(sizeof(node));
    newnode->info = x;
    newnode->next = NULL;
    return newnode;
}

void insertfirst(node **phead, SV x) {
    node* newnode = createnode(x);
    newnode->next = *phead;
    *phead = newnode;
}
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = newnode;
    }
}
```

// Ham xoa sinh vien khoi danh sach (dua tren ma so)

// (Chuc nang nay khong duoc yeu cau trong yeu cau cua ban, nhung van giu lai trong ma)

```
void deletesv(node **phead, char *maso) {
    if (isempty(*phead)) {
        printf("Danh sach rong\n");
        return;
    }
    node *current = *phead, *prev = NULL;
    while (current != NULL) {
        if (strcmp(current->info.maso, maso) == 0) {
            if (prev == NULL) { // Xoa phan tu dau
                *phead = current->next;
            } else {
                prev->next = current->next;
            }
            free(current);
            printf("Da xoa sinh vien co ma so: %s\n", maso);
            return;
        }
        prev = current;
        current = current->next;
    }
    printf("Khong tim thay sinh vien voi ma so: %s\n", maso);
}
```

// Ham xuat danh sach sinh vien co diem trung binh lon hon 5

```
void xuatdanhmuc(node* phead) {
    if (isempty(phead)) {
        printf("Danh sach rong\n");
        return;
    }
    node* current = phead;
    int found = 0;  // Bien kiem tra xem co sinh vien nao co diem trung binh > 5 khong
    while (current != NULL) {
        if (current->info.diemtrungbinh > 5) { // Kiem tra dieu kien diem trung binh
            xuat1sv(current->info);  // Xuat sinh vien thoa man dieu kien
            found = 1;
        }
        current = current->next;
    }
    if (!found) {
        printf("Khong co sinh vien nao co diem trung binh lon hon 5\n");
    }
}
```

//C: Struct tìm kiếm sinh viên

```
// Dinh nghia kieu du lieu sinh vien
typedef struct {
    char maso[11];
    char hoten[41];
    float diemtrungbinh;
} SV;
// Ham nhap thong tin cho mot sinh vien
void nhap1sv(SV *x) {
// Ham xuat thong tin cua mot sinh vien
void xuat1sv(SV x) {
    printf("%s - %s - %.2f\n", x.maso, x.hoten, x.diemtrungbinh);
}

// Ham khoi tao danh sach lien ket don (ban dau la NULL)
void init(node **phead) {
    *phead = NULL;
}
// Ham tao node moi
node* createnode(SV x) {
// Ham kiem tra danh sach rong
int isempty(node* phead) {
    return phead == NULL;
}
// Ham them sinh vien vao dau danh sach
void insertfirst(node **phead, SV x) {
    node* newnode = createnode(x);
    newnode->next = *phead;
    scanf("%s", x->maso);
    getchar(); // De tranh viec nhap bi loi khi dung fgets hay gets sau do
    fgets(x->hoten, 41, stdin);
    x->hoten[strcspn(x->hoten, "\n")] = 0; // Xoa dau '\n' do fgets them vao
    scanf("%f", &x->diemtrungbinh);
}

// Dinh nghia cau truc node cho danh sach lien ket don
typedef struct node {
    SV info;
    struct node* next;
} node;
    node* newnode = (node*)malloc(sizeof(node));
    newnode->info = x;
    newnode->next = NULL;
    return newnode;
}
```

```
    *phead = newnode;
}
```

// Ham them sinh vien vao cuoi danh sach

```
void insertlast(node **phead, SV x) {
    node* newnode = createnode(x);
    if (isempty(*phead)) {
        *phead = newnode;
    } else {
        node* current = *phead;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = newnode;
    }
}
```

// Ham xoa sinh vien khoi danh sach (dua tren ma so)

// (Chuc nang nay khong duoc yeu cau trong yeu cau cua ban, nhung van giu lai trong ma)

```
void deletesv(node **phead, char *maso) {
    if (isempty(*phead)) {
        printf("Danh sach rong\n");
        return;
    }
    node *current = *phead, *prev = NULL;
    while (current != NULL) {
        if (strcmp(current->info.maso, maso) == 0) {
            if (prev == NULL) { // Xoa phan tu dau
                *phead = current->next;
            } else {
                prev->next = current->next;
            }
            free(current);
            printf("Da xoa sinh vien co ma so: %s\n", maso);
            return;
        }
        prev = current;
        current = current->next;
    }
    printf("Khong tim thay sinh vien voi ma so: %s\n", maso);
}
```

// Ham xuat danh sach sinh vien co diem trung binh lon hon 5

```
void xuatdanhmuc(node* phead) {
    if (isempty(phead)) {
        printf("Danh sach rong\n");
        return;
    }
    node* current = phead;
    int found = 0;  // Bien kiem tra xem co sinh vien nao co diem trung binh > 5 khong
    while (current != NULL) {
        if (current->info.diemtrungbinh > 5) { // Kiem tra dieu kien diem trung binh
            xuat1sv(current->info);  // Xuat sinh vien thoa man dieu kien
            found = 1;
        }
        current = current->next;
    }
    if (!found) {
        printf("Khong co sinh vien nao co diem trung binh lon hon 5\n");
    }
}
```

// Ham tim sinh vien theo ma so

```
void timsv(node* phead, char* maso) {
    if (isempty(phead)) {
        printf("Danh sach rong\n");
        return;
    }
    node* current = phead;
    while (current != NULL) {
        if (strcmp(current->info.maso, maso) == 0) {
            xuat1sv(current->info);
            return;
        }
        current = current->next;
    }
    printf("Khong tim thay\n");
}
```

//D: Struct Sắp xếp ds sinh viên

```
typedef struct SinhVien {
    char maSo[11];
    char hoTen[41];
    float diemTrungBinh;
    struct SinhVien* next;
} SinhVien;

SinhVien* taoSinhVien(char* maSo, char* hoTen, float diemTrungBinh) {
    SinhVien* sv = (SinhVien*)malloc(sizeof(SinhVien));
    strcpy(sv->maSo, maSo);
    strcpy(sv->hoTen, hoTen);
    sv->diemTrungBinh = diemTrungBinh;
    sv->next = NULL;
```

```
    return sv;
}
void themSinhVien(SinhVien** head,
SinhVien* sv) {
    if (*head == NULL || (*head)-
>diemTrungBinh < sv->diemTrungBinh) {
        sv->next = *head;
        *head = sv;
    } else {
        SinhVien* current = *head;
        while (current->next != NULL &&
current->next->diemTrungBinh >= sv-
>diemTrungBinh) {
            current = current->next;
        }
        sv->next = current->next;
        current->next = sv;
    }
}
void inDanhSachSinhVien(SinhVien* head)
{
    SinhVien* current = head;
    while (current != NULL) {
        printf("%s - %s - %.2f\n", current-
>maSo, current->hoTen, current-
>diemTrungBinh);
        current = current->next;
    }
}
```

//E: Struct chèn sinh viên vào ds

```
typedef struct SinhVien {
    char maSo[11];
    char hoTen[41];
    float diemTrungBinh;
    struct SinhVien* next;
} SinhVien;
```

// Tạo sinh viên mới

```
SinhVien* taoSinhVien(char* maSo, char*
hoTen, float diemTrungBinh) {
    SinhVien* sv =
(SinhVien*)malloc(sizeof(SinhVien));
    strcpy(sv->maSo, maSo);
    strcpy(sv->hoTen, hoTen);
    sv->diemTrungBinh = diemTrungBinh;
    sv->next = NULL;
    return sv;
}
```

// Thêm sinh viên vào danh sách

```
void themSinhVien(SinhVien** head, SinhVien* sv) {
    if (*head == NULL) {
        *head = sv;
    } else {
```

```
        SinhVien* current = *head;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = sv;
    }
}
```

// In danh sách sinh viên

```
void inDanhSachSinhVien(SinhVien* head)
{
    SinhVien* current = head;
    while (current != NULL) {
        printf("%s - %s - %.2f\n", current->maSo, current->hoTen, current->diemTrungBinh);
        current = current->next;
    }
}
```

// Tìm sinh viên theo mã số

```
SinhVien* timSinhVien(SinhVien* head, char* maSo) {
    SinhVien* current = head;
    while (current != NULL && strcmp(current->maSo, maSo) != 0) {
        current = current->next;
    }
    return current;
}
```

// Thêm sinh viên sau sinh viên có mã số X

```
void themSinhVienSau(SinhVien* head, char* maSo, SinhVien* svMoi) {
    SinhVien* sv = timSinhVien(head, maSo);
    if (sv != NULL) {
        svMoi->next = sv->next;
        sv->next = svMoi;
    } else {
        printf("Not Found\n");
    }
}
```

Bài 5: stack and queue

Câu 3: trên lớp

// Dinh nghia cau truc ngan xep

```
typedef struct {
```

```
    int top;
    unsigned capacity;
    int* array;
} Stack;
```

// Ham tao ngan xep voi dung luong da cho

```
Stack* createStack(unsigned capacity) {
    Stack* stack = (Stack*) malloc(sizeof(Stack));
    stack->capacity = capacity;
```

// Ham kiem tra ngan xep rong

```
int isEmpty(Stack* stack) {
    return stack->top == -1;
}
```

// Ham lay phan tu ra khoi ngan xep

```
int pop(Stack* stack) {
    if (isEmpty(stack))
        return -1;
    return stack->array[stack->top--];
}
```

// Ham chuyen doi tu thap phan sang he bat ky

```
void decToBase(int num, int base) {
    Stack* stack = createStack(MAX);
n");
}
```

```
    stack->top = -1;
    stack->array = (int*) malloc(stack->capacity * sizeof(int));
    return stack;
}
```

// Ham them phan tu vao ngan xep

```
void push(Stack* stack, int item) {
    stack->array[++stack->top] = item;
}
    char digits[] = "0123456789ABCDEF";
    while (num > 0) {
        push(stack, num % base);
        num /= base;
    }
    while (!isEmpty(stack)) {
        printf("%c", digits[pop(stack)]);
    }
    printf("\
```

Câu 4: trên lớp

```
// Dinh nghia cau truc ngan xep
typedef struct {
    int top;
    unsigned capacity;
    int* array;
} Stack;
// Ham tao ngan xep voi dung luong da cho
Stack* createStack(unsigned capacity) {
// Ham kiem tra ngan xep rong
int isEmpty(Stack* stack) {
    return stack->top == -1;
}
// Ham lay phan tu ra khoi ngan xep
int pop(Stack* stack) {
    if (isEmpty(stack))
        return -1;
    return stack->array[stack->top--];
}
// Ham chuyen doi tu thap phan sang he bat ky
void decToBase(int num, int base) {
    Stack* stack = createStack(100);
// Ham tinh tong bang de quy
int tinhTongDeQuy(int n) {
    if (n == 1)
        return 1;
    return n + tinhTongDeQuy(n - 1);
}
    Stack* stack = (Stack*) malloc(sizeof(Stack));
    stack->capacity = capacity;
    stack->top = -1;
    stack->array = (int*) malloc(stack->capacity * sizeof(int));
    return stack;
}
// Ham them phan tu vao ngan xep
void push(Stack* stack, int item) {
    stack->array[++stack->top] = item;
}
    char digits[] = "0123456789ABCDEF";
    while (num > 0) {
        push(stack, num % base);
        num /= base;
    }
    while (!isEmpty(stack)) {
        printf("%c", digits[pop(stack)]);
    }
    printf("\n");
}
```

```
// Ham tinh tong bang phuong phap khu de quy su dung ngan xep
int tinhTongKhuduDeQuy(int n) {
    Stack* stack = createStack(100);
    int tong = 0;
    // Dua cac gia tri vao ngan xep
    while (n > 0) {
        push(stack, n);
        n--;
    }
    // Lay cac gia tri tu ngan xep ra va tinh tong
    while (!isEmpty(stack)) {
        tong += pop(stack);
    }

    return tong;
}
```

Câu 6: Hãy viết code hiện thực hàng đợi theo phương pháp chạy vòng

```
// Định nghĩa cấu trúc của một hàng đợi
typedef struct {
    int items[MAX]; // Mảng lưu trữ các phần tử của hàng đợi
    int front;      // Chỉ số đầu hàng đợi
    int rear;       // Chỉ số cuối hàng đợi
} Queue;
// Hàm kiểm tra hàng đợi có đầy không
int isFull(Queue *q) {
    // Hàng đợi đầy khi (front = 0 và rear = MAX-1) hoặc (front = rear + 1)
    if ((q->front == 0 && q->rear == MAX - 1) || (q->front == q->rear + 1)) {
        return 1; // Hàng đợi đầy
    }
    return 0; // Hàng đợi không đầy
}
// Hàm khởi tạo hàng đợi
void initQueue(Queue *q) {
    q->front = -1; // Đặt chỉ số đầu hàng đợi là -1 (rỗng)
    q->rear = -1;  // Đặt chỉ số cuối hàng đợi là -1 (rỗng)
}

}
// Hàm kiểm tra hàng đợi có rỗng không
int isEmpty(Queue *q) {
    if (q->front == -1) {
        return 1; // Hàng đợi rỗng
    }
    return 0; // Hàng đợi không rỗng
}
```

```
// Hàm thêm một phần tử vào hàng đợi
void enqueue(Queue *q, int value) {
    if (isFull(q)) {
        printf("Hang doi day.\n");
    } else {
        if (q->front == -1) {
            q->front = 0;
        }
        q->rear = (q->rear + 1) % MAX;
        q->items[q->rear] = value;
        printf("Da them phan tu %d vao hang doi.\n", value);
    }
}
// Hàm lấy một phần tử ra khỏi hàng đợi
int dequeue(Queue *q) {
    int value;
    if (isEmpty(q)) {
        printf("Hang doi rong.\n");
        return -1;
    } else {
        value = q->items[q->front];
        if (q->front == q->rear) {
            q->front = -1;
            q->rear = -1;
        } else {
            q->front = (q->front + 1) % MAX;
        }
        printf("Da lay phan tu %d ra khoi hang doi.\n", value);
        return value;
    }
}
// Hàm hiển thị các phần tử trong hàng đợi
void display(Queue *q) {
    int i;
    if (isEmpty(q)) {
        printf("Hang doi rong.\n"); // Thông báo nếu hàng đợi rỗng
    } else {
        printf("Cac phan tu trong hang doi: ");
        for (i = q->front; i != q->rear; i = (i + 1) % MAX) {
            printf("%d ", q->items[i]); // Hiển thị các phần tử trong hàng đợi
        }
        printf("%d\n", q->items[i]); // Hiển thị phần tử cuối cùng
    }
}
```

Câu 7: trên lớp

```
// Dinh nghia cau truc cua mot node
struct Node {
    int data;
    struct Node* next;  };
```

```
// Dinh nghia cau truc cua hang doi
struct Queue {
    struct Node *front, *rear;
};
// Ham tao mot node moi
struct Node* newNode(int k) {
// Ham tao mot hang doi rong
struct Queue* createQueue() {
    struct Queue* q = (struct Queue*)malloc(sizeof(struct Queue));
    q->front = q->rear = NULL;
    return q;
}
// Ham them mot phan tu vao hang doi
void enqueue(struct Queue* q, int k) {
// Ham xoa mot phan tu khoi hang doi
void dequeue(struct Queue* q) {
    if (q->front == NULL)
        return;
    struct Node* temp = q->front;
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = k;
    temp->next = NULL;
    return temp;
}

    struct Node* temp = newNode(k);
    if (q->rear == NULL) {
        q->front = q->rear = temp;
        return;
    }
    q->rear->next = temp;
    q->rear = temp;
}

    q->front = q->front->next;
    if (q->front == NULL)
        q->rear = NULL;
    free(temp);
}
```

Câu 8: Hãy sử dụng ngăn xếp (stack) chạy từng bước bài toán Tháp Hà Nội (không được sử dụng đệ quy).

```
// Cau truc ngan xep (stack)
struct Stack {
    int top;
    int capacity;
    int* array;
};
```

```
// Ham tao ngan xep co dung luong xac dinh
struct Stack* createStack(int capacity) {
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    stack->capacity = capacity;
    stack->top = -1;
    stack->array = (int*)malloc(stack->capacity * sizeof(int));
    return stack;
}
// Ham kiem tra ngan xep co rong khong
int isEmpty(struct Stack* stack) {
    return stack->top == -1;
}
// Ham them phan tu vao ngan xep
void push(struct Stack* stack, int item) {
    stack->array[++stack->top] = item;
}
// Ham lay phan tu ra khoi ngan xep
int pop(struct Stack* stack) {
    return stack->array[stack->top--];
}
// Ham di chuyen dia giua hai coc
void moveDisk(int from, int to, int disk) {
    printf("Chuyen dia %d tu cot %d sang cot %d\n", disk, from, to);
}
// Ham di chuyen dia giua hai ngan xep
void moveDisksBetweenTwoPoles(struct Stack* src, struct Stack* dest, int s, int d) {
    int pole1TopDisk = isEmpty(src) ? -1 : pop(src);
    int pole2TopDisk = isEmpty(dest) ? -1 : pop(dest);
    // Dieu kien de xac dinh cach di chuyen dia
    if (pole1TopDisk == -1) {
        push(src, pole2TopDisk);
        moveDisk(d, s, pole2TopDisk);
    } else if (pole2TopDisk == -1) {
        push(dest, pole1TopDisk);
        moveDisk(s, d, pole1TopDisk);
    } else if (pole1TopDisk > pole2TopDisk) {
        push(src, pole1TopDisk);
        push(src, pole2TopDisk);
        moveDisk(d, s, pole2TopDisk);
    } else {
        push(dest, pole2TopDisk);
        push(dest, pole1TopDisk);
        moveDisk(s, d, pole1TopDisk);
    }
}
```

```
// Ham giai quyet bai toan Thap Ha Noi
void towerOfHanoi(int num_of_disks, struct Stack* src, struct Stack* aux, struct Stack* dest) {
    int total_num_of_moves;
    int s = 1, d = 3, a = 2;
    if (num_of_disks % 2 == 0) {
        int temp = d;
        d = a;
        a = temp;
    }
    total_num_of_moves = (1 << num_of_disks) - 1;
    for (int i = num_of_disks; i >= 1; i--)
        push(src, i);
    for (int i = 1; i <= total_num_of_moves; i++) {
        if (i % 3 == 1)
            moveDisksBetweenTwoPoles(src, dest, s, d);
        else if (i % 3 == 2)
            moveDisksBetweenTwoPoles(src, aux, s, a);
        else if (i % 3 == 0)
            moveDisksBetweenTwoPoles(aux, dest, a, d);
    }
}
```

Bên thực hành

```
//B:Đổi biểu thức trung tố sang hậu tố
// Cấu trúc ngăn xếp
typedef struct {
    char arr[MAX];
    int top;
} Stack;
// Hàm khởi tạo ngăn xếp
void initStack(Stack* stack) {
    stack->top = -1;
}
// Hàm kiểm tra ngăn xếp có rỗng không
int isEmpty(Stack* stack) {
    return stack->top == -1;
}
// Hàm kiểm tra ngăn xếp đầy không
int isFull(Stack* stack) {
    return stack->top == MAX - 1;
}
// Hàm thêm phần tử vào ngăn xếp
void push(Stack* stack, char c) {
    if (isFull(stack)) {
        printf("Stack overflow!\n");
    } else {
        stack->arr[++(stack->top)] = c;
    }
}
```

```
// Hàm lấy phần tử từ ngăn xếp
char pop(Stack* stack) {
    if (isEmpty(stack)) {
        printf("Stack underflow!\n");
        return -1;
    } else {
        return stack->arr[(stack->top)--];
    }
}
// Hàm chuyển đổi từ biểu thức trung tố sang hậu tố
void infixToPostfix(char* infix, char* postfix) {
    Stack stack;
    initStack(&stack);
    int i = 0, j = 0;
    while (infix[i] != '\0') {
        char current = infix[i];
        // Nếu là toán hạng, thêm vào kết quả
        if (isalnum(current)) {
            postfix[j++] = current;
        }
        // Nếu là dấu mở ngoặc, đẩy vào ngăn xếp
        else if (current == '(') {
            push(&stack, current);
        }
        // Nếu là dấu đóng ngoặc, lấy các toán tử từ ngăn xếp đến dấu mở ngoặc
        else if (current == ')') {
```

```
// Hàm kiểm tra độ ưu tiên của toán tử
int precedence(char op) {
    if (op == '+' || op == '-') {
        return 1;
    } else if (op == '*' || op == '/') {
        return 2;
    }
    return 0;
}
            while (!isEmpty(&stack) && stack.arr[stack.top] != '(') {
                postfix[j++] = pop(&stack);
            }
            pop(&stack);  // Loại bỏ dấu '('
        }
        // Nếu là toán tử, xử lý theo độ ưu tiên
        else if (current == '+' || current == '-' || current == '*' || current == '/') {
            while (!isEmpty(&stack) && precedence(stack.arr[stack.top]) >= precedence(current)) {
                postfix[j++] = pop(&stack);
            }
            push(&stack, current);
        }
        i++;
    }
    // Lấy các toán tử còn lại trong ngăn xếp
```

```
    while (!isEmpty(&stack)) {
        postfix[j++] = pop(&stack);
    }
    postfix[j] = '\0';  // Kết thúc chuỗi hậu tố
}
```

//C:Quản lý ds bệnh nhận – Queue

// Cau truc benh nhan

```
typedef struct {
    int ma;         // Ma so benh nhan
    char ten[50];   // Ten benh nhan
    int tuoi;       // Tuoi benh nhan
} BenhNhan;
```

// Bien toan cuc de dem so benh nhan da kham

```
int soBenhNhanDaKham = 0;
```

// Cau truc HangDoi

```
typedef struct {
    BenhNhan benhNhan[MAX];
    int dau;   // Vi tri dau hang doi
    int cuoi;  // Vi tri cuoi hang doi
} HangDoi;
void khoiTaoHangDoi(HangDoi* h) {
    h->dau = -1;
    h->cuoi = -1;
}
```

// Ham khoi tao HangDoi

// Kiem tra HangDoi co rong khong

```
int laRong(HangDoi* h) {
    return h->dau == -1;
}
```

// Kiem tra HangDoi co day khong

```
int laDay(HangDoi* h) {
    return h->cuoi == MAX - 1;
}
```

// Ham them benh nhan vao HangDoi

```
void themBenhNhan(HangDoi* h, BenhNhan b) {
    if (laDay(h)) {
        printf("Hang doi da day! Khong the them benh nhan.\n");
    } else {
        if (h->dau == -1) {  // Neu HangDoi rong, cap nhat dau
            h->dau = 0;
        }
        h->cuoi++;
        h->benhNhan[h->cuoi] = b;
        printf("Benh nhan da duoc them: %s\n", b.ten);
    }
}
```

```
// Ham lay benh nhan tiep theo se kham (Dequeue)
BenhNhan layBenhNhan(HangDoi* h) {
    BenhNhan b = {-1, "", -1};  // Mac dinh benh nhan khong hop le
    if (laRong(h)) {
        printf("Hang doi rong! Khong co benh nhan de lay.\n");
    } else {
        b = h->benhNhan[h->dau];
        for (int i = h->dau; i < h->cuoi; i++) {
            h->benhNhan[i] = h->benhNhan[i + 1];
        }
        h->cuoi--;
        if (h->dau > h->cuoi) {  // Neu HangDoi rong sau khi lay benh nhan
            h->dau = h->cuoi = -1;
        }
        // Tang so benh nhan da kham
        soBenhNhanDaKham++;
        printf("Benh nhan %s dang duoc kham.\n", b.ten);
    }
    return b;
}

// Ham hien thi danh sach benh nhan trong HangDoi
void hienThiHangDoi(HangDoi* h) {
    if (laRong(h)) {
        printf("Khong co benh nhan trong hang doi.\n");
    } else {
        printf("Danh sach benh nhan trong hang doi:\n");
        for (int i = h->dau; i <= h->cuoi; i++) {
            printf("Ma: %d, Ten: %s, Tuoi: %d\n", h->benhNhan[i].ma, h->benhNhan[i].ten, h->benhNhan[i].tuoi);
        }
    }
}

// Ham in so luong benh nhan da kham
void hienThiSoBenhNhanDaKham() {
    printf("So benh nhan da duoc kham: %d\n", soBenhNhanDaKham);
}
```

Bài 6: CAY VA CAY NHI PHAN TIM KIEM

Câu 1: Bên lý thuyết

```
// Cau 1: Khai bao cau truc Node cho cay nhi phan tong quat
typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;   } Node;
```

// Cau 1: Ham tao mot nut moi trong cay

```
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

// Cau 1a: Ham duyet cay theo thu tu NLR (Pre-order)

```
void preOrderTraversal(Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preOrderTraversal(root->left);
        preOrderTraversal(root->right);
    }
}
```

// Cau 1a: Ham duyet cay theo thu tu LNR (In-order)

```
void inOrderTraversal(Node* root) {
    if (root != NULL) {
        inOrderTraversal(root->left);
        printf("%d ", root->data);
        inOrderTraversal(root->right);
    }
}
```

// Cau 1a: Ham duyet cay theo muc (Level-order)

```
void levelOrderTraversal(Node* root) {
    if (root == NULL) return;

    Node* queue[100];
    int front = 0;
    int rear = 0;
    queue[rear++] = root;
```

// Cau 1a: Ham duyet cay theo thu tu LRN (Post-order)

```
void postOrderTraversal(Node* root) {
    if (root != NULL) {
        postOrderTraversal(root->left);
        postOrderTraversal(root->right);
        printf("%d ", root->data);
    }
}
    while (front < rear) {
        Node* current = queue[front++];
        printf("%d ", current->data);
        if (current->left != NULL) queue[rear++] = current->left;
        if (current->right != NULL) queue[rear++] = current->right;
    }
}
```

// Cau 1b: Ham tim nut co gia tri la x

```
Node* searchNode(Node* root, int x) {
    if (root == NULL || root->data == x) return root;
    Node* foundNode = searchNode(root->left, x);
    if (foundNode != NULL) return foundNode;
    return searchNode(root->right, x);
}
```

// Cau 1c: Ham xac dinh chieu cao cua cay

```
int treeHeight(Node* root) {
    if (root == NULL) return 0;
    int leftHeight = treeHeight(root->left);
    int rightHeight = treeHeight(root->right);
    return (leftHeight > rightHeight ? leftHeight : rightHeight) + 1;  }
```

// Cau 1d: Ham dem so luong nut trong cay

```
int countNodes(Node* root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

// Cau 1e: Ham dem so luong nut la cua cay

```
int countLeafNodes(Node* root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) return 1;
    return countLeafNodes(root->left) + countLeafNodes(root->right);
}
```

// Cau 1f: Ham dem so nut du hai cay con

```
int countNodesWithTwoChildren(Node* root) {
    if (root == NULL) return 0;
    int count = 0;
    if (root->left != NULL && root->right != NULL) count = 1;
    return count + countNodesWithTwoChildren(root->left) + countNodesWithTwoChildren(root->right);
}
```

// Cau 1f: Ham dem so nut co gia tri nho hon k

```
int countNodesLessThanK(Node* root, int k) {
    if (root == NULL) return 0;
    int count = (root->data < k) ? 1 : 0;
    return count + countNodesLessThanK(root->left, k) + countNodesLessThanK(root->right, k);
}
```

// Cau 1f: Ham dem so nut co gia tri lon hon nut con trai va nho hon nut con phai

```
int countNodesInRange(Node* root) {
    if (root == NULL) return 0;
    int count = 0;
    if ((root->left != NULL && root->data > root->left->data) && (root->right != NULL && root->data < root->right->data)) {
        count = 1;
    }
    return count + countNodesInRange(root->left) + countNodesInRange(root->right);
}
```

// Cau 1f: Ham dem so nut co chieu cao cay con trai bang chieu cao cay con phai

```
int countNodesWithEqualHeightSubtrees(Node* root) {
    if (root == NULL) return 0;
    int count = 0;
    if (treeHeight(root->left) == treeHeight(root->right)) {
        count = 1;
    }
    return count + countNodesWithEqualHeightSubtrees(root->left) + countNodesWithEqualHeightSubtrees(root->right);
}
```

Câu 2: Bên lý thuyết

// Cau truc cua mot nut trong cay

```
typedef struct Node {
    int value;
    struct Node* left;
    struct Node* right;
} Node;
```

// Ham dem so nut la co gia tri lon hon x

```
int DemNutLaLonHonX(Node* root, int x) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) {
        return (root->value > x) ? 1 : 0;
    }
    return DemNutLaLonHonX(root->left, x) + DemNutLaLonHonX(root->right, x);
}
```

// Ham tim nut co gia tri lon nhat va nho nhat trong cay

```
void TimGiaTriLonNhatVaNhoNhat(Node* root, int* min, int* max) {
    if (root == NULL) return;
    if (root->value < *min) *min = root->value;
    if (root->value > *max) *max = root->value;
    TimGiaTriLonNhatVaNhoNhat(root->left, min, max);
```

// Ham kiem tra cay co can bang chua

```
bool KiemTraCanBang(Node* root) {
    if (root == NULL) return true;
    int leftHeight = ChieuCao(root->left);
    int rightHeight = ChieuCao(root->right);
    if (abs(leftHeight - rightHeight) > 1) return false;
    return KiemTraCanBang(root->left) && KiemTraCanBang(root->right);
}
```

// Ham kiem tra cay can bang hoan toan chua

```
    TimGiaTriLonNhatVaNhoNhat(root->right, min, max);  }
```

// Ham tinh chieu cao cua cay

```
int ChieuCao(Node* root) {
    if (root == NULL) return 0;
    int leftHeight = ChieuCao(root->left);
    int rightHeight = ChieuCao(root->right);
    return (leftHeight > rightHeight) ? leftHeight + 1 : rightHeight + 1;
}
bool LaCanBangHoanToan(Node* root) {
    if (root == NULL) return true;
    if (root->left == NULL && root->right == NULL) return true;
    if (root->left != NULL && root->right != NULL)
        return LaCanBangHoanToan(root->left) && LaCanBangHoanToan(root->right);
    return false;
}
```

// Ham kiem tra xem tat ca cac nut co du hai con chua (tru cac nut la)

```
bool KiemTraDuHaiCon(Node* root) {
    if (root == NULL) return true;
    if (root->left == NULL && root->right == NULL) return true;
    if (root->left != NULL && root->right != NULL)
        return KiemTraDuHaiCon(root->left) && KiemTraDuHaiCon(root->right);
    return false;  }
```

```
// Ham kiem tra xem moi nut tren cay deu nho hon x chua
bool KiemTraMoiNutNhoHonX(Node* root, int x) {
    if (root == NULL) return true;
    if (root->value >= x) return false;
    return KiemTraMoiNutNhoHonX(root->left, x) && KiemTraMoiNutNhoHonX(root->right, x);
}
```

A). Bên thực hành

```
// Khai bao cau truc Node cho cay nhi phan
typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;
// Ham tao mot nut moi trong cay
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
// Ham duyet cay theo thu tu giua (In-order Traversal)
void inOrderTraversal(Node* root) {
    if (root != NULL) {
        inOrderTraversal(root->left); // Duyet cay con ben trai
        printf("%d ", root->data); // In gia tri nut
        inOrderTraversal(root->right); // Duyet cay con ben phai
    }
}
// Ham duyet cay theo thu tu truoc (Pre-order Traversal)
void preOrderTraversal(Node* root) {
    if (root != NULL) {
        printf("%d ", root->data); // In gia tri nut
        preOrderTraversal(root->left); // Duyet cay con ben trai
        preOrderTraversal(root->right); // Duyet cay con ben phai
    }
}
```

// Ham duyet cay theo thu tu sau (Post-order Traversal)

```
void postOrderTraversal(Node* root) {
    if (root != NULL) {
        postOrderTraversal(root->left); // Duyet cay con ben trai
        postOrderTraversal(root->right); // Duyet cay con ben phai
        printf("%d ", root->data); // In gia tri nut
    }
}
```

// Ham dem so luong nut trong cay

```
int countNodes(Node* root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
}
```

// Ham dem so luong nut co gia tri lon hon X

```
int countNodesGreaterThanX(Node* root, int x) {
    if (root == NULL) return 0;
    int count = 0;
    if (root->data > x) count = 1;
    return count + countNodesGreaterThanX(root->left, x) + countNodesGreaterThanX(root->right, x);
```

// Ham dem so luong la trong cay

```
int countLeafNodes(Node* root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) return 1; // Nut la
    return countLeafNodes(root->left) + countLeafNodes(root->right);
}
```

// Ham dem so luong la co gia tri nho hon Y

```
int countLeafNodesLessThanY(Node* root, int y) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) {
        if (root->data < y) return 1; // Nut la co gia tri nho hon Y
        else return 0;
    }
    return countLeafNodesLessThanY(root->left, y) + countLeafNodesLessThanY(root->right, y); // Dem la trong cay con ben trai va phai
}
```

B).Bên thực hành

```
// Dinh nghia cau truc cua mot nut trong cay
typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;
// Ham tao mot nut moi
// Ham khoi tao cay (tra ve NULL)
Node* init() {
    return NULL;
}
// Ham chen mot nut vao cay
Node* insert(Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
// Ham tim gia tri nho nhat trong cay
Node* findMin(Node* root) {
    while (root->left != NULL) root = root->left;
// Ham xoa mot nut khoi cay
Node* removeNode(Node* root, int data) {
    if (root == NULL) return root;
    if (data < root->data)
        root->left = removeNode(root->left, data);
    else if (data > root->data)
        root->right = removeNode(root->right, data);
    else {
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
// Ham kiem tra cay rong
int isEmpty(Node* root) {
    return root == NULL;
}
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);
    return root;
}
    return root;
}
```

```
        // Nut can xoa duoc tim thay
        if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Node* temp = root->left;
            free(root);
            return temp;
        }
        Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = removeNode(root->right, temp->data);
    }
    return root;
}
```

// Ham xuat gia tri cac nut tren cay theo thu tu LNR

```
void inOrderTraversal(Node* root) {
    if (root != NULL) {
        inOrderTraversal(root->left);
        printf("%d ", root->data);
        inOrderTraversal(root->right);
    }
}
```

// Ham xuat gia tri cac nut tren cay theo thu tu NLR

```
void preOrderTraversal(Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preOrderTraversal(root->left);
        preOrderTraversal(root->right);
    }
}
```

```
// Ham xuat gia tri cac nut tren cay theo thu tu LRN
void postOrderTraversal(Node* root) {
    if (root != NULL) {
        postOrderTraversal(root->left);
        postOrderTraversal(root->right);
        printf("%d ", root->data);
    }
}
```

```
// Ham hien thi menu chuc nang
void menu() {
    printf("------ MENU ------\n");
    printf("1. Chen phan tu vao cay\n");
    printf("2. Xoa phan tu khoi cay\n");
    printf("3. Duyet cay theo thu tu LNR\n");
    printf("4. Duyet cay theo thu tu NLR\n");
    printf("5. Duyet cay theo thu tu LRN\n");
    printf("6. Kiem tra cay rong\n");
    printf("7. Thoat\n");
    printf("------------------\n");
}
```

C).Bên thực hành

```
// Dinh nghia cau truc cua mot nut trong cay
typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;
// Ham tao mot nut moi
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
// Ham khoi tao cay (tra ve NULL)
Node* init() {
    return NULL;
}
// Ham kiem tra cay rong
int isEmpty(Node* root) {
    return root == NULL;
}
// Ham chen mot nut vao cay
Node* insert(Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root; }
```

// Ham xuat danh sach tang dan (duyet LNR)

```
void inOrderTraversal(Node* root) {
    if (root != NULL) {
        inOrderTraversal(root->left);
        printf("%d ", root->data);
        inOrderTraversal(root->right);
    }
}
```

// Ham xuat danh sach giam dan (duyet RNL)

```
void inOrderReverseTraversal(Node* root)
{
    if (root != NULL) {
        inOrderReverseTraversal(root->right);
        printf("%d ", root->data);
        inOrderReverseTraversal(root->left);
    }
}
```

// Ham kiem tra cay co phai la cay nhi phan dung khong

```
int isBSTUtil(Node* root, int min, int max)
{
    if (root == NULL)
        return 1;
    if (root->data < min || root->data > max)
        return 0;
    return isBSTUtil(root->left, min, root-
>data - 1) &&
        isBSTUtil(root->right, root->data +
1, max);
}

int isBST(Node* root) {
    return isBSTUtil(root, INT_MIN,
INT_MAX);
}
```

// Ham tim cha cua nut co khoa x

```
Node* findParent(Node* root, int x) {
    if (root == NULL || root->data == x)
        return NULL;
    if ((root->left != NULL && root->left-
>data == x) ||
        (root->right != NULL && root->right-
>data == x))
        return root;
    if (x < root->data)
        return findParent(root->left, x);
    else
        return findParent(root->right, x);
}
```

// Ham dem so nut la

```
int countLeafNodes(Node* root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right
== NULL) return 1;
    return countLeafNodes(root->left) +
countLeafNodes(root->right);
}
```

// Ham dem so nut giua

```
int countInternalNodes(Node* root) {
    if (root == NULL || (root->left == NULL
&& root->right == NULL)) return 0;
    return 1 + countInternalNodes(root->left)
+ countInternalNodes(root->right);
}
```

// Ham dem kich thuoc cua cay (so nut tren cay)

```
int countNodes(Node* root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) +
countNodes(root->right);
}
```

// Ham xac dinh do sau/chieu cao cua cay

```
int treeHeight(Node* root) {
    if (root == NULL) return 0;
    int leftHeight = treeHeight(root->left);
    int rightHeight = treeHeight(root->right);
    return 1 + (leftHeight > rightHeight ?
leftHeight : rightHeight);
}
```

// Ham tim gia tri nho nhat tren cay

```
int findMinValue(Node* root) {
    if (root == NULL) return INT_MAX;
    while (root->left != NULL) root = root-
>left;
    return root->data;
}
```

// Ham tim gia tri lon nhat tren cay

```
int findMaxValue(Node* root) {
    if (root == NULL) return INT_MIN;
    while (root->right != NULL) root = root-
>right;
    return root->data;
}
```

// Ham tinh tong cac gia tri tren cay

```
int sumValues(Node* root) {
    if (root == NULL) return 0;
    return root->data + sumValues(root->left) + sumValues(root->right);
}
```

```
// Ham hien thi menu chuc nang
void menu() {
    printf("------ MENU ------\n");
    printf("1. Chen phan tu vao cay\n");
    printf("2. Xuat danh sach tang dan\n");
    printf("3. Xuat danh sach giam dan\n");
    printf("4. Kiem tra cay nhi phan dung\n");
    printf("5. Tim cha cua nut co khoa x\n");
    printf("6. Dem so nut la\n");
    printf("7. Dem so nut giua\n");
    printf("8. Dem kich thuoc cua cay\n");
    printf("9. Xac dinh do sau cua cay\n");
    printf("10. Tim gia tri nho nhat tren cay\n");
    printf("11. Tim gia tri lon nhat tren cay\n");
    printf("12. Tinh tong cac gia tri tren cay\n");
    printf("13. Thoat\n");
    printf("------------------\n");
}
```

Bài 8: BANG BAM

Câu 1: Bên lý thuyết

```
typedef struct Node {
    char word[50];
    char mean[100];
    struct Node* next;
} Node;
Node* dictionary[MAX];
// Ham hash de anh xa chu cai dau tien cua tu sang dia chi
int hash(char c) {
    return c - 'a';
}
// Ham them tu vao tu dien
void addWord(char* word, char* mean) {
    int index = hash(word[0]);
    Node* newNode = (Node*)malloc(sizeof(Node));
    strcpy(newNode->word, word);
    strcpy(newNode->mean, mean);
    newNode->next = dictionary[index];
    dictionary[index] = newNode;
}
```

```
// Ham xem tu dien theo chu cai dau tien
void viewByLetter(char letter) {
    int index = hash(letter);
    Node* temp = dictionary[index];
    while (temp != NULL) {
        printf("Word: %s, Mean: %s\n", temp->word, temp->mean);
        temp = temp->next;
    }
}
// Ham xem toan bo tu dien
void viewAll() {
    for (int i = 0; i < MAX; i++) {
        Node* temp = dictionary[i];
        while (temp != NULL) {
            printf("Word: %s, Mean: %s\n", temp->word, temp->mean);
            temp = temp->next;
        }
    }
}
// Ham tim kiem tu
Node* searchWord(char* word) {
    int index = hash(word[0]);
    Node* temp = dictionary[index];
    while (temp != NULL) {
        if (strcmp(temp->word, word) == 0) {
            return temp;
        }
        temp = temp->next;
    }
    return NULL;
}
// Ham xoa tu
void deleteWord(char* word) {
    int index = hash(word[0]);
    Node* temp = dictionary[index];
    Node* prev = NULL;
    while (temp != NULL && strcmp(temp->word, word) != 0) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return; // Tu khong ton tai
    if (prev == NULL) {
        dictionary[index] = temp->next;
    } else {
        prev->next = temp->next;
    }
    free(temp);
}
```

```
// Ham xoa toan bo tu dien
void deleteAll() {
    for (int i = 0; i < MAX; i++) {
        Node* temp = dictionary[i];
        while (temp != NULL) {
            Node* toFree = temp;
            temp = temp->next;
            free(toFree);
        }
        dictionary[i] = NULL;
    }
}
// Ham menu
void menu() {
    printf("1. Them tu\n");
    printf("2. Xem tu dien theo chu cai\n");
    printf("3. Xem toan bo tu dien\n");
    printf("4. Tra cuu tu\n");
    printf("5. Xoa tu\n");
    printf("6. Xoa toan bo tu dien\n");
    printf("7. Thoat\n");
}
```

Câu 2: Bên lý thuyết

```
// Dinh nghia cau truc cua mot nut trong bang bam
typedef struct Node {
    char MSSV[10];
    int line;
    struct Node* next; // Con tro den nut tiep theo trong danh sach lien ket
} HashNode;
HashNode* hashTable[TABLE_SIZE]; // Bang bam voi kich thuoc co dinh
// Ham chen mot MSSV moi vao bang bam
void insert(char* MSSV, int line) {
    int index = hash(MSSV); // Tinh chi so bam
    HashNode* newNode = (HashNode*)malloc(sizeof(HashNode)); // Tao nut moi
```

```
// Ham bam de tinh chi so cho MSSV
int hash(char* MSSV) {
    int sum = 0;
    for (int i = 0; MSSV[i] != '\0'; i++) {
        sum += MSSV[i];
    }
    return sum % TABLE_SIZE;
}

    strcpy(newNode->MSSV, MSSV);
    newNode->line = line;
    newNode->next = hashTable[index]; // Chen nut moi vao dau danh sach lien ket
    hashTable[index] = newNode;
}
```

// Ham tim kiem MSSV trong bang bam

```
int search(char* MSSV) {
    int index = hash(MSSV); // Tinh chi so bam
    HashNode* temp = hashTable[index]; // Truy cap danh sach lien ket tai chi so nay
    while (temp != NULL) {
        if (strcmp(temp->MSSV, MSSV) == 0) {
            return temp->line; // Tra ve dong chua thong tin sinh vien neu tim thay
        }
        temp = temp->next;
    }
```

// Ham xem MSSV trong khoang tu n1 den n2

```
void viewMSSVInRange(int n1, int n2) {
    for (int i = 0; i < TABLE_SIZE; i++) {
        HashNode* temp = hashTable[i];
        while (temp != NULL) {
            if (temp->line >= n1 && temp->line <= n2) {
                printf("%s\n", temp->MSSV);
            }
            temp = temp->next;
        }
    }
}
```

```
    return -1; // Khong tim thay
}
```

// Ham xem tat ca MSSV trong bang bam

```
void viewAllMSSV() {
    for (int i = 0; i < TABLE_SIZE; i++) {
        HashNode* temp = hashTable[i];
        while (temp != NULL) {
            printf("%s\n", temp->MSSV);
            temp = temp->next;
        }
    }
}
```

// Ham xem thong tin sinh vien qua MSSV

```
void viewStudentInfo(char* MSSV, FILE* file) {
    int line = search(MSSV); // Tim dong chua MSSV
    if (line == -1) {
        printf("Sinh vien khong ton tai.\n");
        return;
    }
    fseek(file, 0, SEEK_SET); // Di chuyen con tro file ve dau
    char buffer[256];
    int currentLine = 0;
    while (fgets(buffer, sizeof(buffer), file)) {
        if (currentLine == line) {
            printf("%s", buffer); // In ra thong tin sinh vien neu tim thay
            return;
        }
        currentLine++;
    }
}
```

// Ham hien thi menu

```
void menu() {
    printf("1. Xem tat ca MSSV\n");
    printf("2. Xem MSSV trong khoang tu n1 den n2\n");
    printf("3. Xem thong tin sinh vien qua MSSV\n");
    printf("4. Thoat\n");
}
```

Câu 3: Bên lý thuyết

// Ham tai du lieu tu file vao bang bam

```
void loadFileToHashTable(const char* filename) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        printf("Khong mo duoc file.\n");
        return;
    }
    char buffer[256];
    int line = 0;
    while (fgets(buffer, sizeof(buffer), file)) {
        char MSSV[10];
        sscanf(buffer, "%s", MSSV);
        insert(MSSV, line); // Chen MSSV vao bang bam
        line++;
    }
    fclose(file);
}
```

```
typedef struct Node {
    string key;
    string value;
    Node* next;
} Node;
Node* hashTable[TABLE_SIZE];
// Ham bam de tinh chi so cho key
// Ham chen mot key-value moi vao bang bam su dung chaining with replacement
void insert(string key, string value) {
    int index = hash(key);
    Node* newNode = new Node();
    newNode->key = key;
    newNode->value = value;
    newNode->next = NULL;
    if (hashTable[index] == NULL) {
        hashTable[index] = newNode;
    } else {
        Node* temp = hashTable[index];
        if (hash(temp->key) == index) {
            newNode->next = hashTable[index];
            hashTable[index] = newNode;
        } else {
            while (temp->next != NULL &&
hash(temp->next->key) != index) {
                temp = temp->next;
```

```
int hash(string key) {
    int sum = 0;
    for (int i = 0; i < key.length(); i++) {
        sum += key[i];
    }
    return sum % TABLE_SIZE;
}
            }
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }
}
// Ham tim kiem key trong bang bam
string search(string key) {
    int index = hash(key);
    Node* temp = hashTable[index];
    while (temp != NULL) {
        if (temp->key == key) {
            return temp->value;
        }
        temp = temp->next;
    }
    return "Khong tim thay"; // Khong tim
thay
}
```

// Ham xoa key khoi bang bam

```
void deleteNode(string key) {
    int index = hash(key);
    Node* temp = hashTable[index];
    Node* prev = NULL;
    while (temp != NULL && temp->key !=
key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        cout << "Key khong ton tai.\n";
        return;
    }
    if (prev == NULL) {
        hashTable[index] = temp->next;
    } else {
        prev->next = temp->next;
    }
    delete temp;
}
```

// Ham in tat ca cac phan tu trong bang bam

```
void printTable() {
    for (int i = 0; i < TABLE_SIZE; i++) {
        cout << "[" << i << "]: ";
        Node* temp = hashTable[i];
        while (temp != NULL) {
            cout << "(" << temp->key << ", " <<
temp->value << ") -> ";
            temp = temp->next;
        }
        cout << "NULL\n";
    }
}
```

Câu 4: Bên lý thuyết

// Dinh nghia cau truc cua mot nut trong cay BST

```
struct TreeNode {
    string key;
    string value;
    TreeNode* left;
    TreeNode* right;
    TreeNode(string k, string v) : key(k),
value(v), left(NULL), right(NULL) {}
};
```

// Ham chen mot nut moi vao cay BST

```
TreeNode* insertBST(TreeNode* root,
string key, string value) {
    if (!root) return new TreeNode(key,
value);
    if (key < root->key)
        root->left = insertBST(root->left, key,
value);
    else if (key > root->key)
        root->right = insertBST(root->right,
key, value);
    return root; }
```

// Ham tim kiem mot nut trong cay BST

```
TreeNode* searchBST(TreeNode* root,
string key) {
    if (!root || root->key == key) return root;
    if (key < root->key)
        return searchBST(root->left, key);
    return searchBST(root->right, key);
}
```

// Ham xoa mot nut trong cay BST

```
TreeNode* deleteBST(TreeNode* root,
string key) {
    if (!root) return root;
    if (key < root->key)
        root->left = deleteBST(root->left, key);
    else if (key > root->key)
        root->right = deleteBST(root->right,
key);
    else {
        if (!root->left) {
            TreeNode* temp = root->right;
            delete root;
            return temp;
        }
        if (!root->right) {
            TreeNode* temp = root->left;
            delete root;
            return temp;
        }
        TreeNode* temp = root->right;
        while (temp && temp->left)
            temp = temp->left;
        root->key = temp->key;
        root->value = temp->value;
        root->right = deleteBST(root->right,
temp->key);
    }
    return root;
}
```

// Ham in cay BST theo thu tu inorder

```
void inorderBST(TreeNode* root) {
    if (!root) return;
    inorderBST(root->left);
    cout << "(" << root->key << ", " << root-
>value << ") ";
    inorderBST(root->right);
}
#define TABLE_SIZE 10
TreeNode* hashTable[TABLE_SIZE];
```

// Ham bam de tinh chi so cho key

```
int hashFunction(string key) {
    int sum = 0;
    for (size_t i = 0; i < key.length(); ++i) {
        sum += key[i];
    }
    return sum % TABLE_SIZE;
}
```

// Ham chen mot key-value moi vao bang bam

```
void insert(string key, string value) {
    int index = hashFunction(key);
    hashTable[index] = insertBST(hashTable[index], key, value);
}
```

// Ham tim kiem key trong bang bam

```
string search(string key) {
    int index = hashFunction(key);
    TreeNode* node = searchBST(hashTable[index], key);
    if (node)
        return node->value;
    return "Khong tim thay"; // Khong tim thay
}
```

// Ham xoa key khoi bang bam

```
void deleteNode(string key) {
    int index = hashFunction(key);
    hashTable[index] = deleteBST(hashTable[index], key);
}
```

// Ham in tat ca cac phan tu trong bang bam

```
void printTable() {
    for (int i = 0; i < TABLE_SIZE; i++) {
        cout << "[" << i << "]: ";
        inorderBST(hashTable[i]);
        cout << "NULL" << endl;
    }
}
```

A).Bên thực hành

```
typedef struct HashTable {
    int* table;
    int size;
} HashTable;
```

// Ham khoi tao bang bam

```
HashTable* createTable(int size) {
    HashTable* hashTable = (HashTable*)malloc(sizeof(HashTable));
    hashTable->size = size;
    hashTable->table = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        hashTable->table[i] = -1; // Khoi tao cac vi tri bang -1 (gia tri rong)
    }
    return hashTable;
}
```

// Ham bam

```
int hashFunction(int key, int size) {
    return key % size;
}
```

// Ham chen phan tu vao bang bam

```
void insert(HashTable* hashTable, int key) {
    int index = hashFunction(key, hashTable->size);
    while (hashTable->table[index] != -1) {
        index = (index + 1) % hashTable->size; // Do tuyen tinh
    }
    hashTable->table[index] = key;
}
```

// Ham tim kiem phan tu trong bang bam

```
int search(HashTable* hashTable, int key) {
    int index = hashFunction(key, hashTable->size);
    int start = index;
    while (hashTable->table[index] != -1) {
        if (hashTable->table[index] == key) {
            return index; // Tra ve vi tri tim thay
        }
        index = (index + 1) % hashTable->size;
        if (index == start) {
            break; // Da quay lai vi tri ban dau, ket thuc tim kiem
        }
    }
    return -1; // Khong tim thay
}
```

// Ham xoa phan tu trong bang bam

```
void removeKey(HashTable* hashTable, int key) {
    int index = search(hashTable, key);
    if (index != -1) {
        hashTable->table[index] = -1; // Gan gia tri -1 de danh dau la vi tri rong
        // Do tuyen tinh lai de tranh xung dot
        int nextIndex = (index + 1) % hashTable->size;
        while (hashTable->table[nextIndex] != -1) {
            int rehashKey = hashTable->table[nextIndex];
            hashTable->table[nextIndex] = -1;
            insert(hashTable, rehashKey);
            nextIndex = (nextIndex + 1) % hashTable->size;
        }
    }
}
```

// Ham hien thi bang bam

```
void display(HashTable* hashTable) {
    for (int i = 0; i < hashTable->size; i++) {
        if (hashTable->table[i] != -1) {
            printf("%d ", hashTable->table[i]);
        } else {
            printf("_ ");
        }
    }
    printf("\n"); }
```

B).Bên thực hành

```
// Dinh nghia cau truc cua mot nut trong danh sach lien ket
typedef struct Node {
    int data;
    struct Node* next;
} Node;
// Ham khoi tao bang bam
HashTable* createTable(int size) {
    HashTable* hashTable = (HashTable*)malloc(sizeof(HashTable));
    hashTable->size = size;
    hashTable->table = (Node**)malloc(size * sizeof(Node*));
// Ham bam
int hashFunction(int key, int size) {
// Ham chen phan tu vao bang bam
void insert(HashTable* hashTable, int key) {
    int index = hashFunction(key, hashTable->size);
    Node* newNode = (Node*)malloc(sizeof(Node));
// Ham tim kiem phan tu trong bang bam
int search(HashTable* hashTable, int key) {
    int index = hashFunction(key, hashTable->size);
    Node* temp = hashTable->table[index];
    while (temp != NULL) {
        if (temp->data == key) {
            return 1; // Tim thay
        }
        temp = temp->next;
    }
    return 0; // Khong tim thay
}
// Dinh nghia cau truc cua bang bam
typedef struct HashTable {
    Node** table;
    int size;
} HashTable;

    for (int i = 0; i < size; i++) {
        hashTable->table[i] = NULL;
    }
    return hashTable;
}

    return key % size;
}
    newNode->data = key;
    newNode->next = hashTable->table[index];
    hashTable->table[index] = newNode;
}
```

```
// Ham xoa phan tu trong bang bam
void removeKey(HashTable* hashTable, int key) {
    int index = hashFunction(key, hashTable->size);
    Node* temp = hashTable->table[index];
    Node* prev = NULL;
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return; // Khong tim thay phan tu can xoa
    if (prev == NULL) {
        hashTable->table[index] = temp->next; // Xoa nut dau tien
    } else {
        prev->next = temp->next; // Xoa nut khong phai dau tien
    }
    free(temp);
}
// Ham hien thi bang bam
void display(HashTable* hashTable) {
    for (int i = 0; i < hashTable->size; i++) {
        Node* temp = hashTable->table[i];
        printf("Table[%d]: ", i);
        while (temp != NULL) {
            printf("%d -> ", temp->data);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}
```

Bài tập ôn thêm

Câu 1: Ôn tập 1

```
// 1).Dinh nghia cau truc cho sinh vien
typedef struct {
    char MaSV[11];
    char TenSV[31];
    float DiemTB;
} SinhVien;
// Ham tim kiem sinh vien theo ma so
int timKiemSinhVien(SinhVien ds[], int n, char x[]) {
    for (int i = 0; i < n; i++) {
        if (strcmp(ds[i].MaSV, x) == 0) {
            return i; // Tra ve vi tri tim thay
        }
    }
    return -1; // Khong tim thay
}
```

// 2).**Ham hoan doi hai so nguyen**

```
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

// Ham phan doan mang va tra ve vi tri cua phan tu chot

```
int partition(int a[], int left, int right) {
    int pivot = a[right];
    int i = left - 1;
    for (int j = left; j < right; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[right]);
    return i + 1;
}
```

// Ham QuickSort de sap xep mang

```
void QuickSort(int a[], int left, int right) {
    if (left < right) {
        int pivot = partition(a, left, right);
        QuickSort(a, left, pivot - 1);
        QuickSort(a, pivot + 1, right);
    }
}
```

// 3).**Ham sap xep mang tang dan bang giai thuat Insertion Sort**

```
void InsertionSort(int a[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j = j - 1;
        }
        a[j + 1] = key;
    }
}
```

Câu 2:

// Cau 2a: Khai bao cau truc du lieu de bieu dien mot da thuc

```
typedef struct DonThuc {
    int heSo;
    int soMu;
    struct DonThuc* next;
} DonThuc;
```

// Cau 2a: Ham tao mot don thuc moi

```
DonThuc* taoDonThuc(int heSo, int soMu)
{
    DonThuc* newNode =
(DonThuc*)malloc(sizeof(DonThuc));
    newNode->heSo = heSo;
    newNode->soMu = soMu;
    newNode->next = NULL;
    return newNode;
}
```

// Cau 2b: Ham them don thuc vao sau don thuc co so mu le dau tien, neu khong co chen vao dau da thuc

```
void themDonThuc(DonThuc** head, int
heSo, int soMu) {
    DonThuc* newNode =
taoDonThuc(heSo, soMu);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    DonThuc* current = *head;
    DonThuc* prev = NULL;
    while (current != NULL) {
        if (current->soMu % 2 != 0) {
            newNode->next = current->next;
            current->next = newNode;
            return;
        }
        prev = current;
        current = current->next;
    }
    newNode->next = *head;
    *head = newNode;
}
```

// Cau 2c: Ham tinh gia tri cua da thuc theo x

```
int tinhGiaTriDaThuc(DonThuc* head, int
x) {
    int result = 0;
    DonThuc* current = head;
    while (current != NULL) {
        result += current->heSo * pow(x,
current->soMu);
        current = current->next;
    }
    return result;
}
```

// Cau 2d: Ma gia mo ta cac buoc tim node co gia tri lon nhat tinh theo x, tra ve gia tri do

```
BEGIN
  INPUT: head, x
  SET max_value = NEGATIVE_INFINITY
  SET current = head
  WHILE current is not NULL
    SET value = current.heSo * (x^current.soMu)
    IF value > max_value THEN
      SET max_value = value
    END IF
    SET current = current.next
  END WHILE
  RETURN max_value
END
```

// Cau 2e: Ham tim node co gia tri lon nhat tinh theo x, tra ve gia tri do

```
int timGiaTriLonNhat(DonThuc* head, int x) {
    if (head == NULL) {
        return -1; // Hoac gia tri phu thuoc vao bai toan cua ban
    }
    int max_value = head->heSo * pow(x, head->soMu);
    DonThuc* current = head->next;
    while (current != NULL) {
        int value = current->heSo * pow(x, current->soMu);
        if (value > max_value) {
            max_value = value;
        }
        current = current->next;
    }
    return max_value;
}
```

Câu 1: Ôn tập 2

// Cau 1: Su dung cau truc du lieu danh sach lien ket don, khai bao cau truc du lieu de bieu dien da thuc P(x)

```
typedef struct DonThuc {
    int heSo;
    int soMu;
    struct DonThuc* next;
} DonThuc;
```

// Ham tao mot don thuc moi

```
DonThuc* taoDonThuc(int heSo, int soMu)
{
    DonThuc* newNode =
(DonThuc*)malloc(sizeof(DonThuc));
    newNode->heSo = heSo;
    newNode->soMu = soMu;
    newNode->next = NULL;
    return newNode;
}
```

// Ham them don thuc vao da thuc

```
void themDonThuc(DonThuc** head, int
heSo, int soMu) {
    DonThuc* newNode =
taoDonThuc(heSo, soMu);
    newNode->next = *head;
    *head = newNode;
}
```

// Cau 2: Mo ta thuat toan TimBacMax de tim don thuc co bac cao nhat trong da thuc P(x)

```
BEGIN
INPUT: head (danh sach lien ket don bieu
dien da thuc)
SET bacMax = NEGATIVE_INFINITY
SET current = head
WHILE current is not NULL
IF current.soMu > bacMax THEN
SET bacMax = current.soMu
END IF
SET current = current.next
END WHILE
RETURN bacMax
END
```

// Cau 3: Cai dat ham TimBacMax bang ngon ngu lap trinh C

```
int TimBacMax(DonThuc* head) {
    if (head == NULL) {
        return -1; // Hoac gia tri phu thuoc vao
bai toan cua ban
    }
    int bacMax = head->soMu;
    DonThuc* current = head->next;
    while (current != NULL) {
        if (current->soMu > bacMax) {
            bacMax = current->soMu;
        }
        current = current->next;
    }
    return bacMax;
}
```